Script-Controlled ACL: Restrict Record Access Based on Field Value

Ideation Phase: Brainstorm & Idea Prioritization

Date: October 3, 2026	**Maximum Marks:** 4 Marks
Team ID: SWTID-2026-8089	**Project Name:** Script-Controlled ACL

Brainstorming gives every team member an open space to contribute ideas. Quantity is encouraged first, ideas are built upon by others, and the best ones are then grouped and prioritized.

Step 1: Team Gathering, Collaboration and Problem Statement

- **Objective:** Align the team on project scope, decide how we collaborate (shared board and ServiceNow instance), and confirm the core security challenge.
- **Selected Problem Statement:** In many ServiceNow tables, any user with basic access can view, edit, create or delete records, including sensitive ones. Record-level control is needed so that users see only records matching their branch (for example EEE), while administrators keep full access. Without it, student data is exposed to unauthorized users.

Step 2: Brainstorm, Idea Listing and Grouping

- **Technique:** Open ideation for maximum volume of ideas, followed by thematic affinity clustering.

Generated Ideas (Raw List):

#	Idea
1	Create custom roles (bb1 to bb4) and assign them to a test EEE user.
2	Create a custom table, Institution Details (u_institution_details), with Branch as a choice field (ECE, EEE, CSE).
3	Write a READ ACL with a data condition (Branch is EEE) and an advanced script that checks roles.
4	Create separate CREATE, WRITE and DELETE ACLs, each tied to its own role (bb2, bb3, bb4).
5	Give administrators full access through the admin override and an `gs.hasRole('admin')` check in the script.
6	Verify results by impersonating the EEE user, a user without roles, and the admin.
7	Hide sensitive records using only client-side form rules (UI policies).
8	Track who views records manually in a spreadsheet.

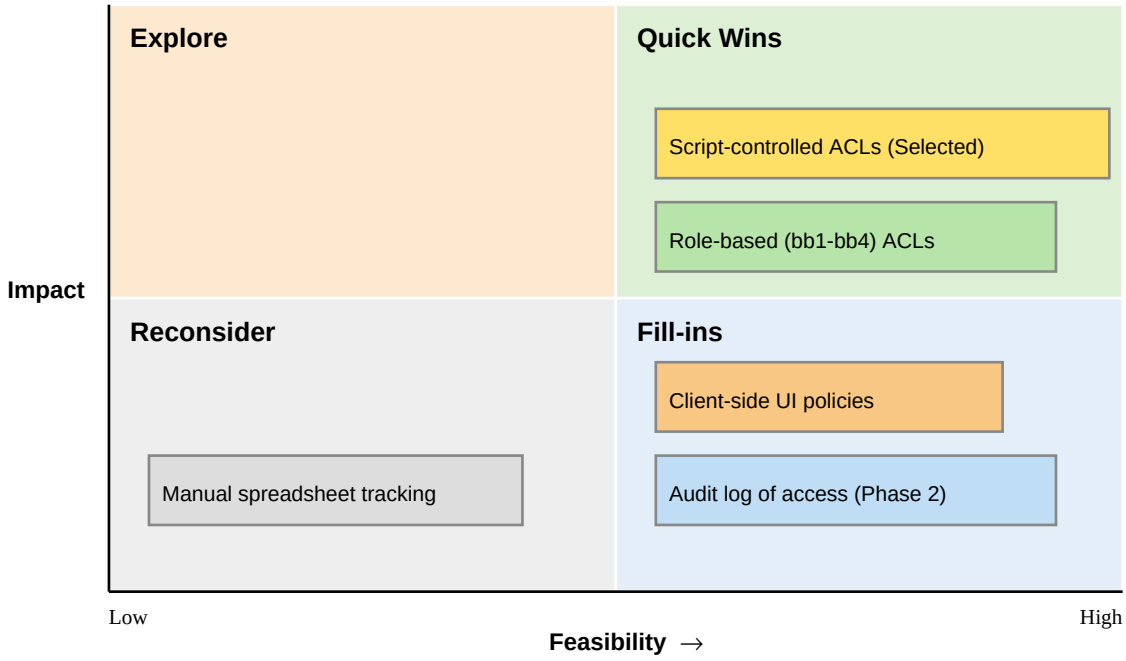
Idea Grouping (Thematic Clusters):

- **Cluster A, Access Setup (Users, Roles, Table):** Ideas #1, #2.
- **Cluster B, ACL Security Rules (Read, Create, Write, Delete):** Ideas #3, #4, #5.
- **Cluster C, Testing and Verification:** Idea #6.
- **Cluster D, Weak / Manual Methods (deprioritized):** Ideas #7, #8. Client-side rules can be bypassed, and manual tracking does not enforce anything.

Step 3: Idea Prioritization

Ideas were mapped on an Impact vs. Feasibility matrix (High, Medium, Low).

Idea Description	Impact	Feasibility	Priority / Action
Script-controlled READ ACL with data condition and role check	High	High	Top Priority (Selected)
Role-based CREATE, WRITE and DELETE ACLs (bb2, bb3, bb4)	High	High	Selected
Impersonation-based testing of all three user types	Medium	High	Selected (Validation)
Audit log of record access	Medium	Medium	Phase 2
Client-side UI policies only	Low	High	Rejected (not secure)
Manual spreadsheet tracking	Low	Low	Backlog



Alignment with Project Milestones

- **Milestone 1:** Creation of users and roles (EEE User, bb1 to bb4) and the Institution Details table with sample records.
- **Milestone 2:** Creation of READ, CREATE, WRITE and DELETE ACLs, followed by verification.
- **Expected Outcome:** bb1 users see only EEE records, users without roles see nothing, and admins see all records. With bb1 to bb4 together, a user can view, create, edit and delete EEE records.